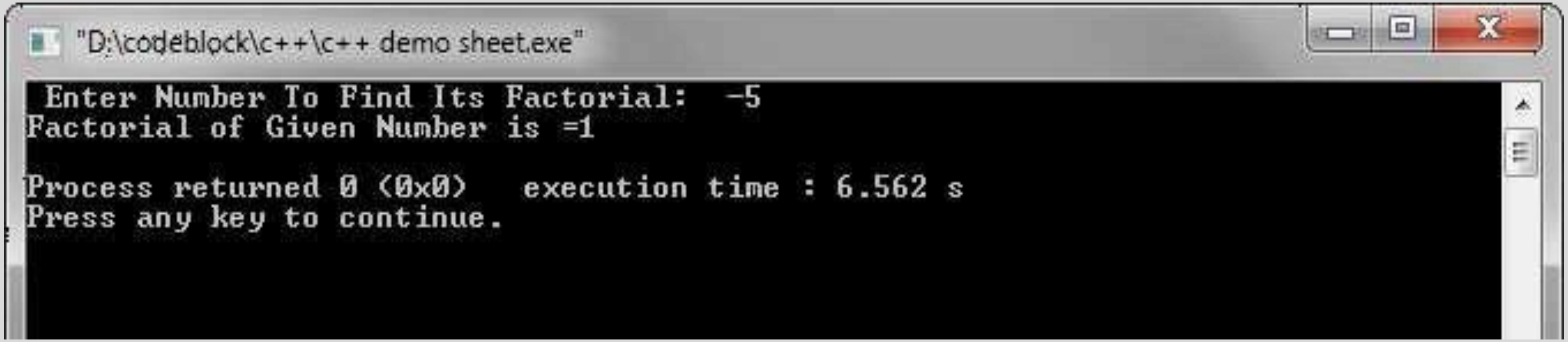
Subject- Object Oriented Programming (CO212)
Unit 4 – Templates and Exception Handling
Topic – 4.4 Exception Handling

WHAT IS AN EXCEPTION ?

- It is often easier to write a program by first assuming that nothing incorrect will happen
- But sometimes user enters an Invalid value e.g. In a factorial program if a user enters any negative value.



- For such cases Exception Handling is used.
- Using Exception Handling you can easily manage and respond to run-time error.

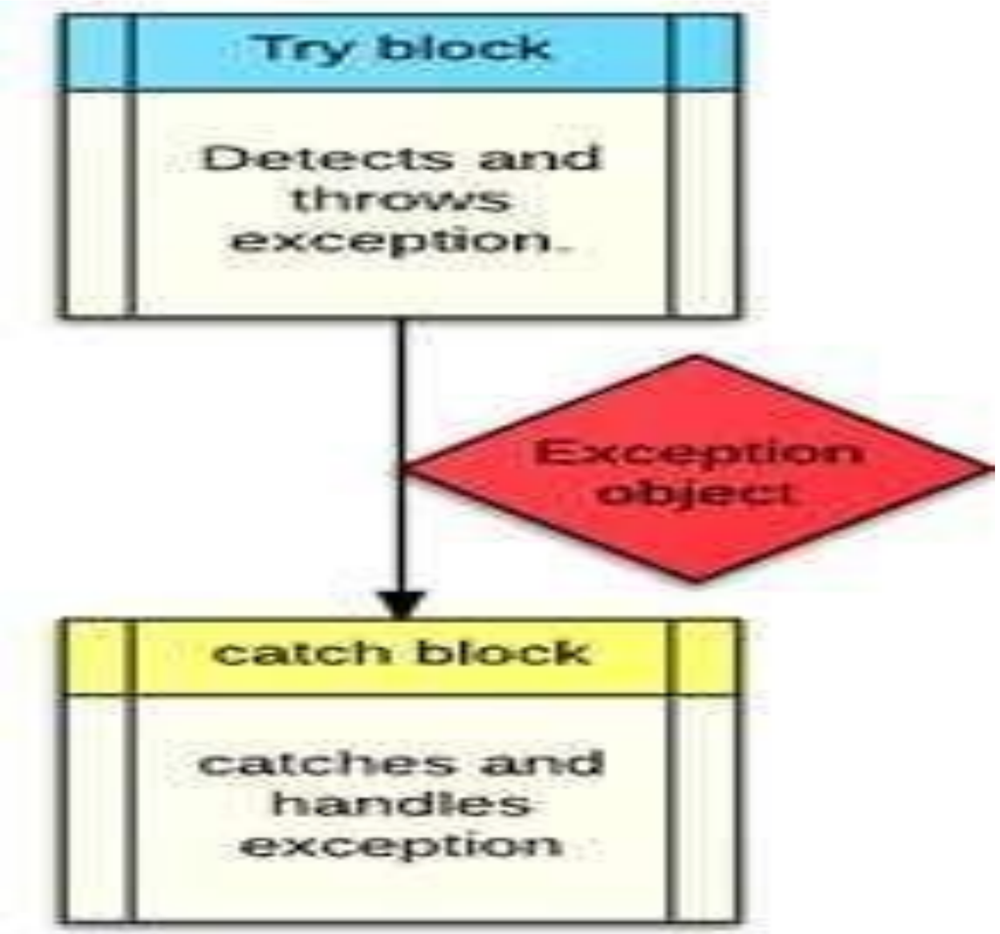
COMMON FAILURES

- `new` not allocating memory
- Division by zero
- Invalid function parameters

- C does not have Exception Handling Mechanism
- But, C++ have build in Exception Handling Mechanism
- We can handle exception without exception handling feature but that will not be a professional and systematic approach and it will not clear the program logic.

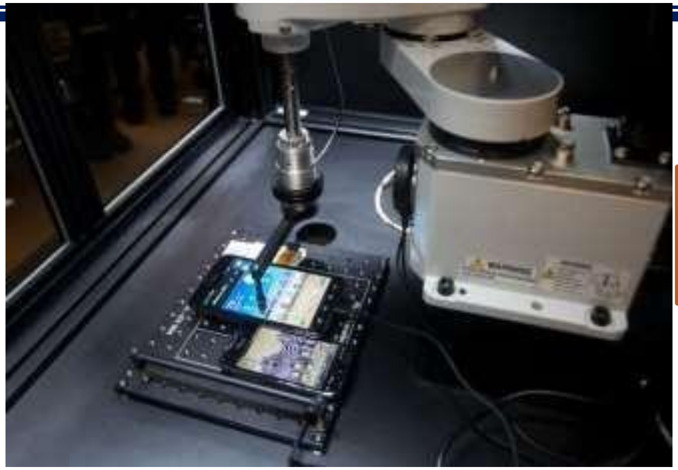
Try , Catch , Throw

- In Exception handling we use three keywords try , catch and throw.
- Program statements you want to monitor for exceptions are contained in a try block.
- If an exception occurs within the try block it is thrown by throw keyword
- And thrown exception is caught using Catch and processed.



RELATING EXCEPTION HANDLING WITH REAL LIFE

try



If a Smartphone fails the test

throw

Resend to production

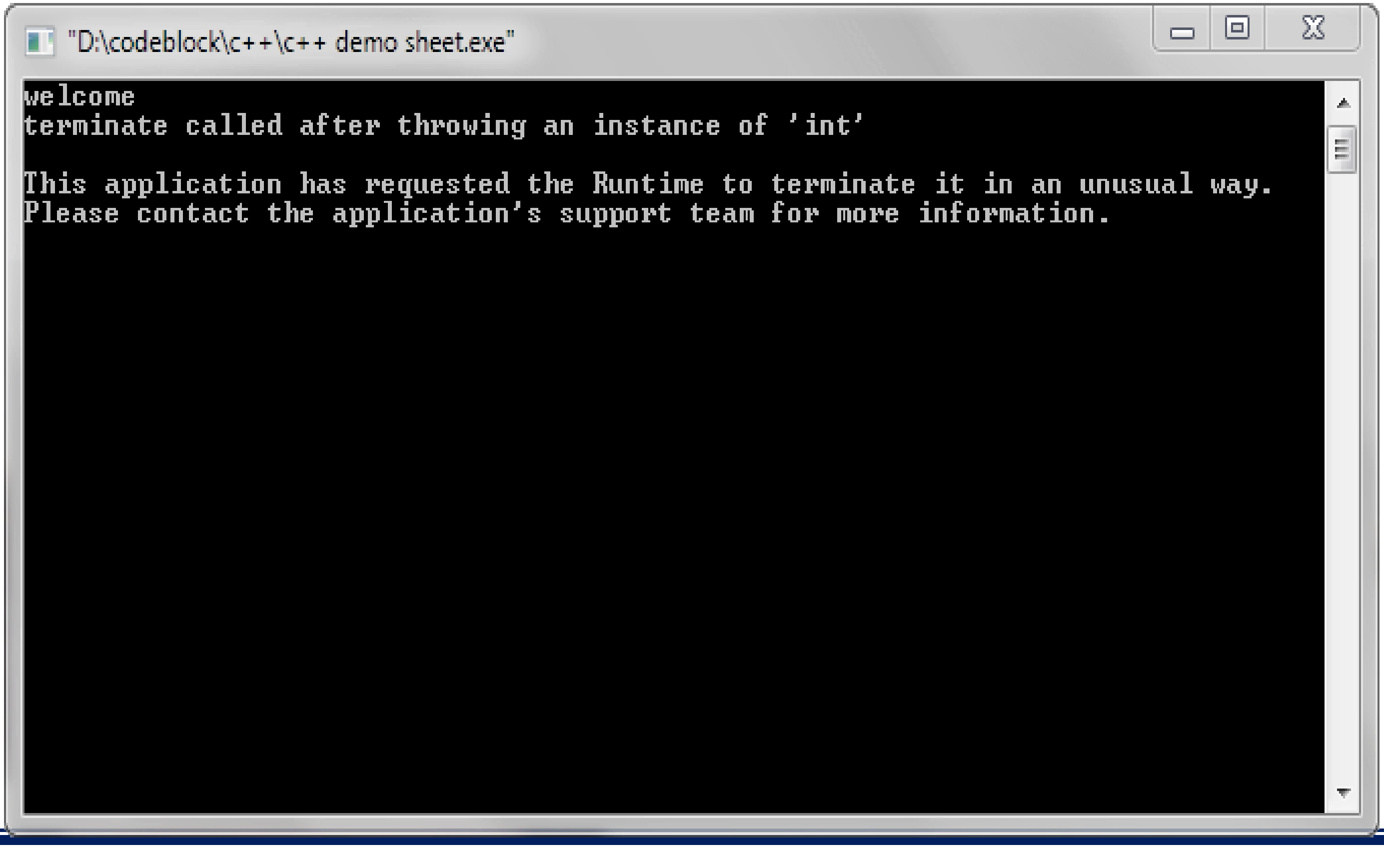
catch

IMPORTANT POINTS

- When a try block end it must be followed by a catch block.
- If we write catch block but don’t write try block then the program will show error.
- If we don’t write try and catch but write throw then the program will build and run but it will show run time error.

```
#include<iostream>
int main()
{

std::cout<<"welcome";
  throw 10;
  return 0;
}
```



Exception Handling Example

```
int main( )
{
   int a,b;
   cout<<"enter values of a & b:";
    cin>>a>>b;
   int x=a-b;
    try{
          if (x!=0)
          {
          cout<<"result(a/x)="<<a/x;
          }
          else
          {
          throw(x);
          }
   }
          catch(int i)
          {
          cout<<"exception caught: x = “ << x;
          }
          cout<<"end";
          return 0;
       }
```

"D:\codeblock\c++\c++ demo sheet.exe"

```
enter values of a and b
10 10
exception caught:x=0
end
Process returned 0 (0x0)   execution time : 10.065 s
Press any key to continue.
```

Try-Catch Block

- In a program a block of code which may produce error is placed in try block.
- When an exception is detected in try it is thrown using throw statement.

Syntax:

```
try
{
        throw exception;
}
catch(type argument)
{


}
```

THROWING MECHANISM

- An exception is thrown using the throw statement in one of the following ways:

```
throw(exception);
throw exception;
throw;
```

- We can use multiple throw statements using if – else statement.

e.g.

```
int i=3;
try {
   if(i==1) throw 1;
    if(i==2) throw 2;
}
```

CATCHING MECHANISM

- A catch block look like a function definition and is of the form

```
catch (type arg)
    {
        // statements for managing exceptions
    }
```

- The type indicates which type of exception that catch block handles
- The catch statement catches an exception whose type matches with the type of catch argument.
- When it is caught the code in the catch block gets executed
- Due to mismatched, if an exception is not caught, abnormal program termination will occur
- The catch block is simply skipped if the catch statement does not catch an exception

MULTIPLE CATCH STATEMENT

- Syntax for multiple catch statement:

```
try
{

}
catch (ExceptionType name)
{

}
catch (ExceptionType name)
{

}
finally
{

}
```

Multiple catch

- In multiple catch when an exception is thrown the exception handler searches for an appropriate match.
- When no match is found the program is terminated.
- It is possible that arguments of several catch statement matches the type of an exception, in such a case the first handler that matches the exception is executed.

Thank You..